

Advanced Programming Language (Java)

Object-Oriented Programming: Classes, Objects, Non-Access Modifiers, Inheritance, Abstraction & Interfaces

Course Code: C-COS 202

Dr. Babatunde I. Ayinla

Faculty of Computing
Department of Computer Science
University of Ibadan

August 29, 2026

Course Overview

- **Course:** Advanced Programming Language (Java)
- **Course Code:** C-COS 202
- **Lecturer:** Dr. Babatunde I. Ayinla
- **Department:** Computer Science, Faculty of Computing
- **Institution:** University of Ibadan

Weekly Breakdown

- 1 Week 5: Classes, Objects and Attributes
- 2 Week 6: Non-Access Modifiers (`final`, `static`)
- 3 Week 7: Inheritance in Java
- 4 Week 8: Abstraction in Java
- 5 Week 9: Interfaces in Java
- 6 Week 10: Advanced Interface Features (JDK 8/9) & Review

Outline

- 1 Week 5: Classes, Objects and Attributes
- 2 Week 6: Non-Access Modifiers
- 3 Week 7: Inheritance in Java
- 4 Week 8: Abstraction in Java
- 5 Week 9: Interfaces in Java
- 6 Week 10: Advanced Interface Features (JDK 8/9) & Review
- 7 Wrap-Up

Week 5: Learning Outcomes

By the end of this week, students should be able to:

- Explain the relationship between a class and an object
- Create Java classes with fields and methods
- Instantiate objects using the `new` keyword
- Access and modify object attributes using dot notation

What is a Class?

Definition

A **class** is a blueprint for creating objects. It encapsulates **fields** (state) and **methods** (behaviour) that are common to all objects of that type.

- In Java, an object is created *from* a class.
- After defining a class, objects are created from it using the `new` keyword.
- Objects hold **state** and can perform **actions**.

Creating an Object – Example 1

Example

Create an object called `myObj` and print the value of `x`.

```
1 public class ClassDemo {  
2     int x = 5;  
3  
4     public static void main(String[] args) {  
5         ClassDemo myObj = new ClassDemo();  
6         System.out.println(myObj.x);  
7     }  
8 }
```

Output

5

Creating an Object – Example 2

```
1  class Student {  
2      int matricNum; // instance variable  
3      String name; // instance variable  
4  
5      public static void main(String args[]) {  
6          Student new_student_1 = new Student();  
7          Student new_student_2 = new Student();  
8          System.out.println(new_student_1.matricNum);  
9          System.out.println(new_student_1.name);  
0          System.out.println(new_student_2.matricNum);  
1          System.out.println(new_student_2.name);  
2      }  
3  }
```

Example 2 – Explanation

Explanation

The code defines a `Student` class with two instance variables, `matricNum` and `name`, and creates two objects to demonstrate default values. It shows that each object of a class holds its own state and can access the class's fields independently.

Java Class Attributes

- Variables declared inside a class are called **attributes** (also known as **fields**).
- Attributes are variables that *belong to a class*.

Example

Create a class Main with two attributes: x and y.

```
1 public class Main {  
2     int x = 5;  
3     int y = 3;  
4 }
```

Accessing Attributes

- Attributes are accessed by creating an object of the class and using **dot syntax** (.).

```
1 public class ClassDemo {  
2     int x = 5;  
3  
4     public static void main(String[] args) {  
5         ClassDemo myObj = new ClassDemo();  
6         System.out.println(myObj.x);  
7     }  
8 }
```

Modifying Attributes

```
1 public class Main {  
2     int x;  
3  
4     public static void main(String[] args) {  
5         Main myObj = new Main();  
6         myObj.x = 40;  
7         System.out.println(myObj.x);  
8     }  
9 }
```

Output

40

Overriding Existing Values

```
1 public class Main {  
2     int x = 10;  
3  
4     public static void main(String[] args) {  
5         Main myObj = new Main();  
6         myObj.x = 25; // x is now 25  
7         System.out.println(myObj.x);  
8     }  
9 }
```

Output

25

Week 5 Summary

- A class is a blueprint; an object is an instance of a class.
- Objects are created with the `new` keyword.
- Fields declared in a class are called attributes.
- Attributes are accessed and modified with dot notation.

Week 5: Exercises & Assignments

- 1 Define, in your own words, the difference between a class and an object, giving one real-world analogy.
- 2 Write a Java class `Book` with attributes `title`, `author`, and `price`. Create two `Book` objects and print their attributes.
- 3 Modify Example 2 (`Student` class) so that it also stores a `department` attribute, then create three student objects.
- 4 Given a class `Car` with attribute `int speed = 0;`, write code that creates a `Car` object and changes its speed to 80.
- 5 **Assignment:** Design a class `Employee` with at least four attributes. Create an array of five `Employee` objects, assign values to each, and print them in a formatted table.
- 6 **Assignment:** Explain, with code, what happens if you try to access an attribute of an object that has not been instantiated (i.e. is `null`).

Week 6: Learning Outcomes

By the end of this week, students should be able to:

- Distinguish access modifiers from non-access modifiers
- Use the `final` modifier on attributes and classes
- Use the `static` modifier on methods and attributes
- Explain `abstract`, `transient`, `synchronized` and `volatile` at a conceptual level

Non-Access Modifiers

Definition

Non-access modifiers do not control visibility (like `public` or `private`); instead they add **other features** to classes, methods, and attributes.

- Most commonly used: `final`, `static`, and `abstract`.

The final Modifier

If you don't want the ability to override an existing attribute value, declare the attribute as `final`.

```
1 public class FinalClass {  
2     final int x = 10;  
3     final double PI = 3.14;  
4  
5     public static void main(String[] args) {  
6         FinalClass myObj = new FinalClass();  
7         myObj.x = 50; // Error: cannot assign to a final variable  
8         myObj.PI = 25; // Error: cannot assign to a final variable  
9         System.out.println(myObj.x);  
0     }  
1 }
```

The static Modifier

A **static method** belongs to the class, not to any specific object – it can be called without creating an object.

```
1 public class StaticClass {  
2     static void myStaticMethod() {  
3         System.out.println("Static methods can be called "  
4             + "without creating objects");  
5     }  
6  
7     public static void main(String[] args) {  
8         myStaticMethod(); // direct call  
9         StaticClass.myStaticMethod(); // via class name  
10    }  
11 }
```

Explanation

A static method belongs to the class itself. It can be called without an object, but it cannot use instance variables or instance methods.

Non-Access Modifiers for Classes

Modifier	Description
<code>final</code>	The class cannot be inherited by other classes.
<code>abstract</code>	The class cannot be used to create objects directly; it must be inherited from.

Non-Access Modifiers for Attributes & Methods

Modifier	Description
<code>final</code>	Attributes and methods cannot be overridden or modified.
<code>static</code>	Belongs to the class, not to objects; all objects share the same static attribute, and static methods can be called without creating an object.
<code>abstract</code>	Only used inside an abstract class, and only on methods. The method has no body – e.g. <code>abstract void run();</code> . The body is supplied by the subclass.
<code>transient</code>	Attributes/methods are skipped when serializing the containing object.
<code>synchronized</code>	Methods can only be accessed by one thread at a time.
<code>volatile</code>	The attribute's value is never cached thread-locally; it is always read from main memory.

Week 6 Summary

- Non-access modifiers add behaviour, not visibility control.
- `final`: locks a value / prevents subclassing.
- `static`: ties a member to the class rather than an instance.
- Other modifiers (`abstract`, `transient`, `synchronized`, `volatile`) fine-tune inheritance, serialization and concurrency behaviour.

Week 6: Exercises & Assignments

- 1 Write a `final` class `Circle` with a `final double PI = 3.14159;` attribute. Explain what happens if a subclass tries to extend it.
- 2 Create a class `MathUtils` with a static method `square(int n)` that returns n^2 . Call it without creating an object.
- 3 Add a static counter attribute to a class `Counter` that increments every time a new object is created; print the counter after creating four objects.
- 4 Explain, with an example, why an instance method cannot directly call a static-only helper without qualification, and vice versa.
- 5 **Assignment:** Research and write a one-page note (or short Java demo) contrasting `synchronized` and `volatile`, including a scenario where each is appropriate.
- 6 **Assignment:** Implement a `BankAccount` class where the interest rate is a `final static double` shared by all accounts, and show that no object can change it.

Week 7: Learning Outcomes

By the end of this week, students should be able to:

- Explain inheritance and the `extends` keyword
- Identify superclass/subclass relationships
- Distinguish single, multilevel, hierarchical, multiple (via interfaces) and hybrid inheritance
- Apply the Java IS-A relationship

Definition

Inheritance is a core OOP concept that allows a class to acquire properties and behaviours from another class, promoting code reusability and better organisation.

- A subclass can reuse the fields and methods of the parent class without rewriting the code.
- A subclass can add its own fields/methods, or modify existing ones, to extend functionality.

Inheritance – Runtime Polymorphism Example

```
1 class Animal {  
2     void sound() { System.out.println("Animal makes a sound"); }  
3 }  
4 class Dog extends Animal {  
5     void sound() { System.out.println("Dog barks"); }  
6 }  
7 class Cat extends Animal {  
8     void sound() { System.out.println("Cat meows"); }  
9 }  
0 class Cow extends Animal {  
1     void sound() { System.out.println("Cow moos"); }  
2 }  
3 public class Geeks {  
4     public static void main(String[] args) {  
5         Animal a;  
6         a = new Dog(); a.sound();  
7         a = new Cat(); a.sound();  
8         a = new Cow(); a.sound();  
9     }  
0 }
```

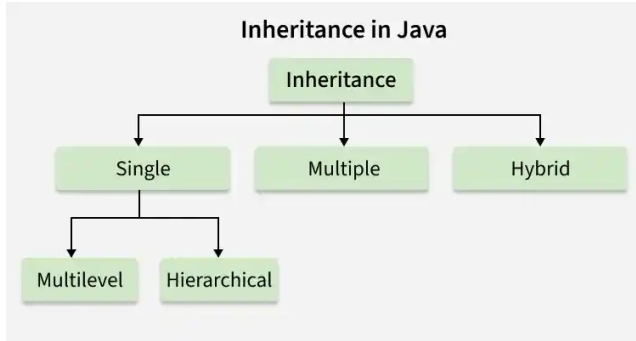
Explanation

Animal is the base class; Dog, Cat and Cow extend it and override sound(). The Geeks class demonstrates runtime polymorphism through method overriding: at runtime, Java calls the overridden version that matches the object's actual type.

```
1  class Parent {  
2      // fields and methods  
3  }  
4  class Child extends Parent {  
5      // additional fields and methods  
6  }
```

Key terms: Class (blueprint), Superclass (properties are inherited), Subclass (inherits another class), extends (keyword used to inherit).

Types of Inheritance in Java



Defines the different ways a class can inherit properties and behaviour from one or more classes.

1. Single Inheritance

A sub-class is derived from only *one* super class.

```
1  class Vehicle {  
2      Vehicle() { System.out.println("This is a Vehicle"); }  
3  }  
4  class Car extends Vehicle {  
5      Car() { System.out.println("This Vehicle is Car"); }  
6  }  
7  public class Test {  
8      public static void main(String[] args) {  
9          Car obj = new Car();  
10     }  
11 }
```

2. Multilevel Inheritance

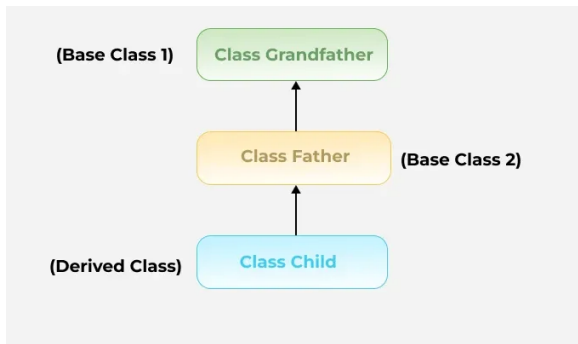


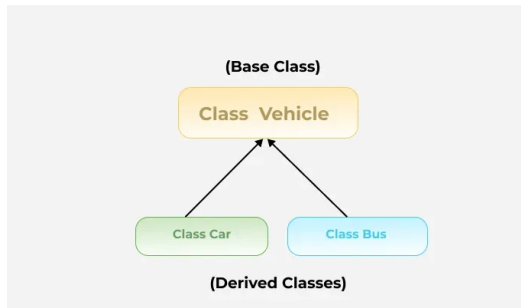
Figure 1: Multiple Inheritance

A derived class inherits a base class, and that derived class also acts as the base class for another class.

Multilevel Inheritance – Code

```
1  class Vehicle {  
2      Vehicle() { System.out.println("This is a Vehicle"); }  
3  }  
4  class FourWheeler extends Vehicle {  
5      FourWheeler() { System.out.println("4 Wheeler Vehicles"); }  
6  }  
7  class Car extends FourWheeler {  
8      Car() { System.out.println("This 4 Wheeler Vehicle is a Car"); }  
9  }  
0  public class Geeks {  
1      public static void main(String[] args) {  
2          Car obj = new Car(); // triggers all constructors in order  
3      }  
4  }
```

3. Hierarchical Inheritance



More than one subclass is derived from a single base class, e.g. cars and buses are both vehicles.

Hierarchical Inheritance – Code

```
1  class Vehicle {  
2      Vehicle() { System.out.println("This is a Vehicle"); }  
3  }  
4  class Car extends Vehicle {  
5      Car() { System.out.println("This Vehicle is Car"); }  
6  }  
7  class Bus extends Vehicle {  
8      Bus() { System.out.println("This Vehicle is Bus"); }  
9  }  
0  public class Test {  
1      public static void main(String[] args) {  
2          Car obj1 = new Car();  
3          Bus obj2 = new Bus();  
4      }  
5  }
```

4. Multiple Inheritance (Through Interfaces)

Explanation

Java does **not** support multiple inheritance with classes. Multiple inheritance is only achieved through **interfaces**.

```
1 interface LandVehicle {
2     default void landInfo() {
3         System.out.println("This is a LandVehicle");
4     }
5 }
6 interface WaterVehicle {
7     default void waterInfo() {
8         System.out.println("This is a WaterVehicle");
9     }
10 }
11 class AmphibiousVehicle implements LandVehicle, WaterVehicle {
12     AmphibiousVehicle() {
13         System.out.println("This is an AmphibiousVehicle");
14     }
15 }
```

5. Hybrid Inheritance

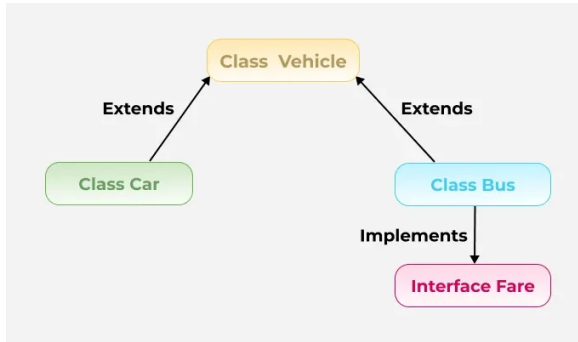


Figure 2: Hybrid Inheritance

A mix of two or more inheritance types. In Java, hybrid inheritance is achieved through interfaces when multiple inheritance is involved.

Hybrid Inheritance – Code (1/2)

```
1  class Vehicle {  
2      void vehicleType() { System.out.println("This is a Vehicle"); }  
3  }  
4  interface Fare {  
5      default void fareInfo() { System.out.println("Fare information"); }  
6  }  
7  class Car extends Vehicle {  
8      void carType() { System.out.println("This is a Car"); }  
9  }  
0  class Bus extends Vehicle implements Fare {  
1      void busType() { System.out.println("This is a Bus"); }  
2  }
```

Hybrid Inheritance – Code (2/2)

```
1 public class GFG {  
2     public static void main(String[] args) {  
3         Car car = new Car();  
4         car.vehicleType(); // inherited  
5         car.carType(); // specific to Car  
6  
7         Bus bus = new Bus();  
8         bus.vehicleType(); // inherited  
9         bus.busType(); // specific to Bus  
10        bus.fareInfo(); // from Fare interface  
11    }  
12 }
```

Explanation

Car extends Vehicle is single inheritance; Bus extends Vehicle *and* implements Fare is hybrid inheritance (a combination of single + interface-based inheritance).

The Java IS-A Relationship

IS-A represents an inheritance relationship: *this* object is a type of *that* object.

```
1 public class SolarSystem { }  
2 public class Earth extends SolarSystem { }  
3 public class Mars extends SolarSystem { }  
4 public class Moon extends Earth { }
```

- SolarSystem is the superclass of Earth and Mars.
- Earth and Mars are subclasses of SolarSystem.
- Moon is a subclass of both Earth and (transitively) SolarSystem.

What Can Be Done in a Subclass?

- Use inherited fields directly, like any other field.
- Declare new fields not present in the superclass.
- Use inherited methods directly, as they are.
- Override an inherited instance method with the same signature.
- Hide an inherited static method with the same signature.
- Declare new methods not present in the superclass.
- Write a subclass constructor that invokes the superclass constructor, implicitly or via `super`.

Advantages and Limitations of Inheritance

Advantages

- Code reusability
- Enables abstraction
- Supports class hierarchy
- Enables polymorphism

Limitations

- Complexity: deep hierarchies are hard to follow
 - keep hierarchies simple or prefer composition
- Tight coupling: superclass changes can break subclasses – prefer loose coupling (interfaces/composition)

- Inheritance lets a subclass reuse a superclass's members.
- Java classes support single, multilevel and hierarchical inheritance directly.
- Multiple and hybrid inheritance require interfaces.
- The IS-A relationship models these hierarchies.

Week 7: Exercises & Assignments

- 1 Draw (on paper) the inheritance diagram for `SolarSystem`, `Earth`, `Mars` and `Moon`, and state which relationships are IS-A.
- 2 Implement multilevel inheritance for `Employee` → `Manager` → `SeniorManager`, each adding one new field.
- 3 Implement hierarchical inheritance where `Shape` is the base class and `Triangle`, `Square`, and `Pentagon` are subclasses.
- 4 Using two interfaces `Flyable` and `Swimmable` (each with a default method), create a class `Duck` that implements both.
- 5 **Assignment:** Build a hybrid-inheritance example for a `SmartPhone` class hierarchy (class + at least two interfaces), and explain in comments which inheritance type each relationship represents.
- 6 **Assignment:** Discuss, in a short write-up, why Java disallows multiple inheritance with classes but allows it with interfaces (the "diamond problem").

Week 8: Learning Outcomes

By the end of this week, students should be able to:

- Define abstraction and explain its purpose
- Create and use abstract classes and abstract methods
- Explain the difference between abstract and concrete methods
- Justify when to use abstraction in program design

What is Abstraction?

Definition

Abstraction is the process of hiding internal implementation details and showing only essential functionality to the user. It focuses on *what* an object does rather than *how* it does it.

- Hides complex details; shows only essential features.
- Abstract classes may declare methods without implementation – subclasses must implement them.
- Changes to implementation do not affect code that depends only on the abstraction.

Achieved via **abstract classes** or **interfaces**.

- **Abstract class:** a restricted class that cannot be instantiated directly.
- **Abstract method:** only usable in an abstract class; it has no body – the body is supplied by the subclass.

Ways to Achieve Abstraction

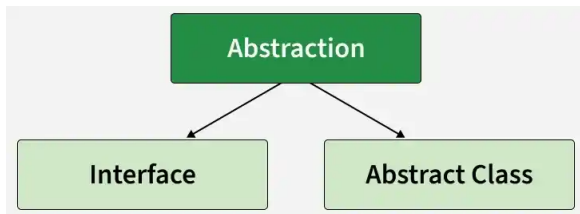


Figure 3: Abstraction

- Abstract Classes (partial abstraction)
- Interfaces (abstraction for behaviour; may contain default or static methods)

Real-Life Example: TV Remote

A TV remote is a classic example of abstraction – users press a button without knowing how the TV works internally.

```
1  abstract class TV {
2      abstract void turnOn();
3      abstract void turnOff();
4  }
5  class TVRemote extends TV {
6      @Override void turnOn() { System.out.println("TV is turned ON."); }
7      @Override void turnOff() { System.out.println("TV is turned OFF."); }
8  }
9  public class Geeks {
10     public static void main(String[] args) {
11         TV remote = new TVRemote();
12         remote.turnOn();
13         remote.turnOff();
14     }
15 }
```

Abstract Class – Definition

Definition

An abstract class is declared using the `abstract` keyword and cannot be instantiated directly. It may contain both abstract methods (no body) and concrete methods (with a body), providing common functionality while leaving some behaviour to subclasses.

- Can contain abstract *and* concrete methods.
- Can have constructors, instance variables, and static members.

Real-World Example: Shape

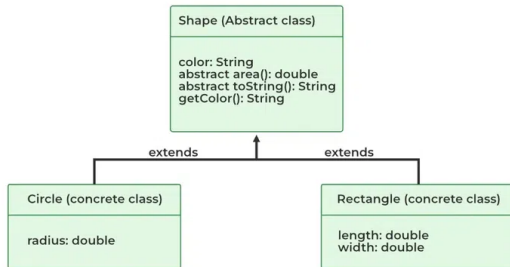


Figure 4: Abstraction

Shape defines common properties/methods. Circle and Rectangle extend it and add their own specific properties (radius, or length/width). `area()` and `toString()` are abstract, so each concrete subclass supplies its own implementation.

Abstract Class – Shape Example (1/2)

```
1  abstract class Shape {
2      String color;
3      abstract double area();
4      public abstract String toString();
5      public Shape(String color) {
6          System.out.println("Shape constructor called");
7          this.color = color;
8      }
9      public String getColor() { return color; }
10 }
11 class Circle extends Shape {
12     double radius;
13     public Circle(String color, double radius) {
14         super(color);
15         this.radius = radius;
16     }
17     @Override double area() { return Math.PI * Math.pow(radius, 2); }
18     @Override public String toString() {
19         return "Circle color is " + super.getColor()
20             + " and area is : " + area();
21     }
22 }
```

Abstract Class – Shape Example (2/2)

```
1 class Rectangle extends Shape {  
2     double length, width;  
3     public Rectangle(String color, double length, double width) {  
4         super(color);  
5         this.length = length;  
6         this.width = width;  
7     }  
8     @Override double area() { return length * width; }  
9     @Override public String toString() {  
10         return "Rectangle color is " + super.getColor()  
11         + " and area is : " + area();  
12     }  
13 }  
14 public class Test {  
15     public static void main(String[] args) {  
16         Shape s1 = new Circle("Red", 2.2);  
17         Shape s2 = new Rectangle("Yellow", 2, 4);  
18         System.out.println(s1.toString());  
19         System.out.println(s2.toString());  
20     }  
21 }
```

Abstract Class with Regular Methods

An abstract class can mix abstract and regular (concrete) methods.

```
1  abstract class Animal {  
2      public abstract void animalSound();  
3      public void sleep() { System.out.println("Zzz"); }  
4  }  
5  class Pig extends Animal {  
6      public void animalSound() {  
7          System.out.println("The pig says: wee wee");  
8      }  
9  }  
10 class Main {  
11     public static void main(String[] args) {  
12         Pig myPig = new Pig();  
13         myPig.animalSound();  
14         myPig.sleep();  
15     }  
16 }
```

Explanation

`Animal myObj = new Animal();` would generate a compile error – an abstract class cannot be instantiated directly.

Advantages of Abstraction

- Makes complex systems easier to understand by hiding implementation detail.
- Keeps different parts of a system separated.
- Maintains code more efficiently.
- Increases security by exposing only necessary detail.

Limitations of Abstraction

- Unnecessary abstraction adds complexity.
- Can make debugging harder – implementation is hidden.
- May require extra classes/interfaces, increasing code volume.
- Too many abstraction layers hurt comprehension.
- Designing good abstraction requires careful planning.
- Adds little benefit in small or simple programs.

- Abstraction hides implementation, exposes essential behaviour.
- An abstract class cannot be instantiated; it may hold both abstract and concrete members.
- Subclasses supply bodies for inherited abstract methods.
- Use abstraction to improve security, maintainability, and clarity – but avoid over-abstracting small programs.

Week 8: Exercises & Assignments

- 1 Explain why `new Animal()` fails when `Animal` is declared abstract, using the compiler's perspective.
- 2 Create an abstract class `Employee` with an abstract method `calculateSalary()`. Implement two subclasses, `FullTimeEmployee` and `PartTimeEmployee`, each with its own salary formula.
- 3 Extend the `Shape` example to add a `Triangle` subclass with its own `area()` and `toString()` implementations.
- 4 Write an abstract class `PaymentMethod` with a concrete method `printReceipt()` and an abstract method `processPayment(double amount)`. Implement it for `CardPayment` and `CashPayment`.
- 5 **Assignment:** Compare abstract classes and interfaces (which you will study next week) in a short table: when would you choose one over the other?
- 6 **Assignment:** Model a `Vehicle` abstraction for an online car-rental system with at least three concrete subclasses, and justify your design choices in comments.

Week 9: Learning Outcomes

By the end of this week, students should be able to:

- Define a Java interface and explain its purpose
- Implement one or more interfaces in a class
- Explain the relationship between a class and an interface
- Apply interfaces to real-world modelling problems

Definition

An interface defines a contract that specifies the behaviour a class should provide. A class *implements* an interface and supplies implementations for its abstract methods. Interfaces can also contain default and static methods with implementations.

- A class uses the `implements` keyword to implement an interface.
- A class can implement multiple interfaces, supporting multiple inheritance of type.
- All variables in an interface are `public`, `static`, and `final` by default.

Interface Example

```
1 interface Shape {  
2     double calculateArea();  
3 }  
4 class Circle implements Shape {  
5     private double r;  
6     public Circle(double r) { this.r = r; }  
7     public double calculateArea() { return Math.PI * r * r; }  
8 }  
9 class Rectangle implements Shape {  
0     private double length, width;  
1     public Rectangle(double length, double width) {  
2         this.length = length; this.width = width;  
3     }  
4     public double calculateArea() { return length * width; }  
5 }  
6 public class Main {  
7     public static void main(String[] args) {  
8         Shape cir = new Circle(5.0);  
9         Shape rect = new Rectangle(4.0, 6.0);  
0         System.out.println("Area of Circle: " + cir.calculateArea());  
1         System.out.println("Area of Rectangle: " + rect.calculateArea());  
2     }  
3 }
```

Interface Example – Explanation

Explanation

Shape declares `calculateArea()`. Circle and Rectangle implement the interface and provide their own area calculations. The references `cir` and `rect` let the program use both objects through the same interface type – this is **dynamic method dispatch**: Java decides which method to call at runtime.

Class vs. Interface

Use a class when...

- You need to represent a real-world entity with attributes and behaviours.
- You need objects that hold state and perform actions.
- You need a template for objects with specific functionality/properties.

Use an interface when...

- You need a contract for behaviour that multiple classes can implement.
- You want to achieve abstraction and multiple inheritance.

Explanation

A class can extend another class, and an interface can extend another interface. Only a class can implement an interface; an interface cannot implement a class.

Modelling with Interfaces – Vehicles (1/2)

```
1 interface Vehicle {  
2     void changeGear(int a);  
3     void speedUp(int a);  
4     void applyBrakes(int a);  
5 }  
6 class Bicycle implements Vehicle {  
7     int speed, gear;  
8     @Override public void changeGear(int newGear) { gear = newGear; }  
9     @Override public void speedUp(int inc) { speed += inc; }  
0     @Override public void applyBrakes(int dec) { speed -= dec; }  
1     public void printStates() {  
2         System.out.println("speed: " + speed + " gear: " + gear);  
3     }  
4 }
```

Modelling with Interfaces – Vehicles (2/2)

```
1 class Bike implements Vehicle {
2     int speed, gear;
3     @Override public void changeGear(int newGear) { gear = newGear; }
4     @Override public void speedUp(int inc) { speed += inc; }
5     @Override public void applyBrakes(int dec) { speed -= dec; }
6     public void printStates() {
7         System.out.println("speed: " + speed + " gear: " + gear);
8     }
9 }
10
11 class Main {
12     public static void main(String[] args) {
13         Bicycle bicycle = new Bicycle();
14         bicycle.changeGear(2); bicycle.speedUp(3); bicycle.applyBrakes(1);
15         bicycle.printStates();
16
17         Bike bike = new Bike();
18         bike.changeGear(1); bike.speedUp(4); bike.applyBrakes(3);
19         bike.printStates();
20     }
21 }
```

Multiple Interfaces

To implement multiple interfaces, separate them with a comma.

```
1 interface FirstInterface {  
2     void myMethod();  
3 }  
4 interface SecondInterface {  
5     void myOtherMethod();  
6 }  
7 class DemoClass implements FirstInterface, SecondInterface {  
8     public void myMethod() { System.out.println("Some text.."); }  
9     public void myOtherMethod() { System.out.println("Some other text..."); }  
0 }
```

Multiple Inheritance Using Interfaces

```
1 interface Add { int add(int a, int b); }
2 interface Sub { int sub(int a, int b); }
3
4 class Cal implements Add, Sub {
5     public int add(int a, int b) { return a + b; }
6     public int sub(int a, int b) { return a - b; }
7 }
8 class GFG {
9     public static void main(String[] args) {
10         Cal x = new Cal();
11         System.out.println("Addition : " + x.add(2, 1));
12         System.out.println("Subtraction : " + x.sub(2, 1));
13     }
14 }
```

Output

```
Addition : 3
Subtraction : 1
```

- An interface is a contract: classes implement its methods.
- Interface variables are implicitly `public static final`.
- A class can implement several interfaces, giving Java multiple inheritance of type.
- Interfaces enable dynamic method dispatch and flexible, decoupled designs.

Week 9: Exercises & Assignments

- 1 Define an interface `Payable` with method `double amountDue()`. Implement it for `Invoice` and `Subscription` classes.
- 2 Implement the `Shape` interface example with an added `Triangle` class.
- 3 Create two interfaces `Readable` and `Writable`, and a class `FileHandler` that implements both.
- 4 Extend the `Vehicle` interface example with a `Truck` class that also tracks `loadCapacity`.
- 5 **Assignment:** Design a small library-management interface hierarchy (e.g. `Borrowable`, `Returnable`) and implement it for `Book` and `DVD` classes.
- 6 **Assignment:** Explain, with a short demo, why interface variables cannot be reassigned after declaration.

Week 10: Learning Outcomes

By the end of this week, students should be able to:

- Use default and static methods in interfaces (JDK 8)
- Recognise functional interfaces
- Use private methods in interfaces (JDK 9)
- Extend one interface from another
- Consolidate all OOP concepts covered in the course

JDK 8: Default Methods

- Interfaces can define methods with default implementations.
- Useful for adding new methods to interfaces without breaking existing implementing classes.

```
1 interface TestInterface {  
2     final int a = 10;  
3     default void display() { System.out.println("hello"); }  
4 }  
5 class TestClass implements TestInterface {  
6     public static void main(String[] args) {  
7         TestClass t = new TestClass();  
8         t.display();  
9     }  
10 }
```

JDK 8: Static Methods

- Interfaces can include static methods.
- Called directly using the interface name; *not* inherited by implementing classes.

```
1 interface TestInterface {  
2     final int a = 10;  
3     static void display() { System.out.println("hello"); }  
4 }  
5 class TestClass implements TestInterface {  
6     public static void main(String[] args) {  
7         TestInterface.display();  
8     }  
9 }
```

JDK 8: Functional Interfaces

- A functional interface has exactly one abstract method.
- Can be used with lambda expressions or method references.
- `@FunctionalInterface` is optional but recommended.

```
1  @FunctionalInterface
2  public interface Calculator {
3      int compute(int x, int y); // single abstract method
4  }
```

JDK 9: Private Methods in Interfaces

- Interfaces can include private methods.
- Private methods are for internal use only – not accessible to, or overridable by, implementing classes.

```
1 interface Vehicle {  
2     private void startEngine() {  
3         System.out.println("Engine started.");  
4     }  
5     default void drive() {  
6         startEngine();  
7         System.out.println("Vehicle is now driving.");  
8     }  
9 }  
0 class Car implements Vehicle { }  
1 public class Main {  
2     public static void main(String[] args) {  
3         Car car = new Car();  
4         car.drive();  
5     }  
6 }
```

Extending Interfaces

One interface can inherit another using `extends`. A class implementing the child interface must implement *all* methods in the inheritance chain.

```
1 interface A {  
2     void method1();  
3     void method2();  
4 }  
5 interface B extends A {  
6     void method3();  
7 }  
8 class GFG implements B {  
9     public void method1() { System.out.println("Method 1"); }  
0     public void method2() { System.out.println("Method 2"); }  
1     public void method3() { System.out.println("Method 3"); }  
2 }
```

JDK 8/9 Interface Features – Summary Table

Feature	Behaviour
Default methods (Java 8)	Provide a body; inherited by implementing classes; can be overridden.
Static methods (Java 8)	Called via interface name; not inherited.
Functional interfaces (Java 8)	Exactly one abstract method; usable with lambdas.
Private methods (Java 9)	Internal helper logic for default/ static methods; not inherited, not overridable.
extends between interfaces	Child interface inherits all parent abstract methods.

Course Review: Key Concepts

- **Classes & Objects:** blueprint vs. instance; attributes; new keyword.
- **Non-Access Modifiers:** final, static, abstract, transient, synchronized, volatile.
- **Inheritance:** single, multilevel, hierarchical, multiple (via interfaces), hybrid; IS-A relationship.
- **Abstraction:** abstract classes and methods; hiding implementation detail.
- **Interfaces:** contracts, multiple implementation, default/static/private methods, functional interfaces.

Week 10: Exercises & Assignments

- ➊ Add a default method `honk()` to the `Vehicle` interface from Week 9 without breaking `Bicycle` or `Bike`.
- ➋ Write a functional interface `StringTransformer` with method `String transform(String s)`, then use it with a lambda expression to convert a string to upper case.
- ➌ Refactor the `Vehicle drive()` example to add a second private helper method, `checkFuelLevel()`, called from `drive()`.
- ➍ Extend interface `A` (with `method1`, `method2`) into interface `B` (adding `method3`) into interface `C` (adding `method4`), and implement `C` fully in one class.
- ➎ **Assignment (Capstone):** Design and implement a small Java console application (e.g. a school records system or an online store) that must use: at least one abstract class, at least two interfaces (one with a default method), at least one example of multilevel or hierarchical inheritance, and at least one static and one final member. Submit source code with comments explaining each OOP concept used.
- ➏ **Assignment:** Write a two-page reflective report comparing abstract classes and interfaces as design tools, using examples from this course, and recommend guidelines for choosing between them in a real project.

Questions?

Dr. Babatunde I. Ayinla
Department of Computer Science, Faculty of Computing
University of Ibadan

Advanced Programming Language (Java) – C-COS 202